Лабораторна робота №5

З дисципліни “Системи програмування”

На тему:“Рефлексія (System.Reflection) та динамічна ідентифікація типів“

Студента 4 курсу: Групи
МІТ-41 Демиденко Андрій

Київ - 2026р.

1. МЕТА ТА ЗАВДАННЯ РОБОТИ

Мета: Опанувати механізми інтроспекції та пізнього зв'язування (Late Binding) платформи .NET за допомогою простору імен `System.Reflection`. Дослідити структуру метаданих збірки та реалізувати алгоритм динамічного вилучення інформації про конструктори, властивості (включно з їхніми поточними значеннями) та методи об'єкта під час виконання програми (Runtime).

Завдання (Варіант 6): Створити клас `Doctor`, який містить мінімум 4 властивості різних типів (одна з яких є колекцією), 3 довільні методи та 2 конструктори (з параметрами та без). Розробити графічний інтерфейс для візуалізації ієрархічної структури (метаданих) екземпляра цього класу.

2. ОПИС АРХІТЕКТУРНОЇ РЕАЛІЗАЦІЇ

Модель даних (Domain Model): Створено цільовий клас `Doctor`.

- **Властивості:** `FullName` (string), `ExperienceYears` (int), `IsAvailable` (bool), `Specialties` (List).
- **Конструктори:** Базовий (ініціалізує значення за замовчуванням) та перевантажений (повна ініціалізація стану).
- **Методи:** `AddSpecialty()`, `DiagnosePatient()`, `ToggleAvailability()`.

Графічний інтерфейс (Avalonia UI): Для відображення розгалуженої структури метаданих використано компонент `TreeView`. Це дозволяє користувачеві інтерактивно розгортати та згортати вузли з конструкторами, властивостями та методами, що відповідає патерну відображення ієрархічних даних.

```
namespace DynamicTypeIdentification
{
    4 references
    public class Doctor
    {
        3 references
        public string FullName { get; set; }
        2 references
        public int ExperienceYears { get; set; }
        4 references
        public bool IsAvailable { get; set; }
        3 references
        public List<string> Specialties { get; set; }

        0 references
        public Doctor()
        {
            FullName = "Невідомо";
            ExperienceYears = 0;
            IsAvailable = false;
            Specialties = new List<string>();
        }

        1 reference
        public Doctor(string fullName, int experienceYears, bool isAvailable, List<string> specialties)
        {
            FullName = fullName;
            ExperienceYears = experienceYears;
            IsAvailable = isAvailable;
            Specialties = specialties;
        }
```

3. ТЕХНІЧНИЙ АНАЛІЗ АЛГОРИТМУ РЕФЛЕКСІЇ

Динамічний аналіз типу здійснюється в обробнику події натискання кнопки через виклик `targetDoctor.GetType()`, який повертає екземпляр `System.Type` — вхідну точку до таблиць метаданих CLR.

1. **Аналіз конструкторів (`GetConstructors`):** За допомогою методу `GetConstructors()` вилучено масив `ConstructorInfo`. Для відновлення точної сигнатури кожного конструктора використано LINQ-запит для ітерації по `GetParameters()`.

2. **Аналіз властивостей та стану (`GetProperties` та `GetValue`):** Вилучено масив `PropertyInfo`. Для отримання фактичних даних з оперативної пам'яті викликано метод `GetValue(targetDoctor)`. *Критична архітектурна умова:* Реалізовано перевірку типів через зіставлення з шаблоном (`value is IEnumerable collection && value is not string`). Це необхідно, оскільки тип `String` неявно реалізує `IEnumerable<char>`. Без жорсткої фільтрації (`is not string`) рефлексія б ітерувала кожен рядок посимвольно, розбиваючи ім'я лікаря на окремі літери у дереві інтерфейсу.

3. **Аналіз методів (`GetMethods`):** Вилучено масив `MethodInfo`. Для ізоляції методів застосовано дві стадії фільтрації:

    - **Фільтрація успадкування:** Використано бітову маску `BindingFlags.DeclaredOnly`, що відсікає базові методи класу `System.Object` (`ToString`, `GetHashCode`, `Equals`).

- **Фільтрація аксесорів:** Використано перевірку властивості `method.IsSpecialName`. Оскільки кожна C#-властивість компілюється у пару прихованих методів (`get_FullName`, `set_FullName`), ця перевірка гарантує вилучення зі списку згенерованого компілятором "візуального сміття", залишаючи лише явно оголошені методи бізнес-логіки.

```
Type targetType = targetDoctor.GetType();
List<TreeViewItem> globalRootNodes = new List<TreeViewItem>();

TreeViewItem ctorsRoot = new TreeViewItem { Header = "[ КОНСТРУКТОРИ ]", IsExpanded = true };
List<TreeViewItem> ctorsList = new List<TreeViewItem>();

ConstructorInfo[] constructors = targetType.GetConstructors(BindingFlags.Public | BindingFlags.Instance);
foreach (ConstructorInfo ctor in constructors)
{
    // Форматування списку параметрів за допомогою LINQ
    string paramSignature = string.Join(", ", ctor.GetParameters().Select(p => $"{p.ParameterType.Name} {p.Name}"));
    ctorsList.Add(new TreeViewItem { Header = $"ctor Doctor({paramSignature})" });
}
```

ВИСНОВОК

У ході виконання лабораторної роботи №5 було успішно досліджено архітектуру метаданих платформи .NET та практично застосовано механізми рефлексії (`System.Reflection`). Розроблено програмний інструмент з графічним інтерфейсом на базі Avalonia UI, здатний динамічно виконувати інтроспекцію невідомих типів об'єктів під час виконання програми.

Експериментально доведено ефективність використання пізнього зв'язування для аналізу топології класів, вилучення сигнатур конструкторів, методів та поточних значень властивостей. Опрацьовано механізми низькорівневої фільтрації членів класу за допомогою бітових масок `BindingFlags` та ідентифікації згенерованих компілятором методів-аксесорів. Здобуті навички є критично важливими для розуміння принципів роботи сучасних ORM-систем, серіалізаторів та фреймворків мета-програмування.